01 / ARCHITECTURE

Three apps. One connected experience.

Project A hosts the todo list and live summary. Projects B and C independently control their own counters.

Project A

HOST + STATE OWNER

Todo list
Live B and C values
Federated counter state
localhost:4200

Project B

COUNTER FRONTEND

Increase, decrease, reset B
Loads state code from A
localhost:4201

Project C

COUNTER FRONTEND

Increase, decrease, reset C
Loads state code from A
localhost:4202

HOW THE PIECES CONNECT

Native Federation loads frontend code between the apps. **Angular signals** refresh the UI. **A-origin browser storage bridges** synchronize counters across tabs.

Frontend only: counter values stay in browser localStorage. No counter API, database, or backend microservice.

02 / APPLICATION FLOW

From startup to a live counter update

Federation provides code loading; browser messaging and storage provide data synchronization.

STARTUP Initialize federation > bootstrap Angular > load or reuse A's counter state > read saved values > render the screen

1. Click +

B calls increment()

2. Send action

update('b', 'increment')

3. Save value

Bridge locks and writes

4. Refresh UI

A and B show the result

WHY A HIDDEN BRIDGE?

Different ports have different storage origins. Each app embeds a hidden page from A, so its bridge can access the same A-origin storage. Parents exchange messages with their bridges using postMessage().

HOW OTHER TABS RECEIVE UPDATES

The writing bridge replies immediately. Browser storage events notify the other bridges, which forward the current pair of values. Signals refresh their parent screens. Web Locks serialize simultaneous updates.

Example: B = 5. Click Increase in B. The bridge stores B = 6; B and A's live summary display 6. C is unchanged.

A clear walkthrough for your audience

Show the independent apps first, then demonstrate how changes appear in the host application.

DEMONSTRATION

1. Start A, B, and C on ports 4200, 4201, and 4202.
2. Open all three localhost URLs in one browser profile.
3. Increase B. Observe B's value change in A.
4. Decrease or reset C. Observe C's value change in A.
5. Reload: counters persist. Add, complete, and delete a todo in A.

WHERE THE LOGIC LIVES

Federation initialization

Each project: src/main.ts / initFederation()

Remote loading

B/C: src/app/app.ts / ngOnInit() and loadRemoteModule()

Counter state

A: src/app/counter-state.ts / getCounterState() and update()

Cross-tab synchronization

A: public/counter-bridge.js / message and storage listeners

Todo behavior

A: src/app/todo.ts / addTask(), toggleTask(), removeTask()

DATA & OPERATING NOTES

Counters persist in browser storage; todo items remain in component memory and clear on reload. Use Chrome or Edge with storage enabled. A must be available for standalone B/C. Different browser profiles or computers have separate values.